

Tutorial: Recording GPS Routes in the Background on Android

CS5520 Final Tutorial Report Prepared by: An Nguyen **Course:** CS5520 — Mobile Application Development, Northeastern University (Summer 2026) **Reference app:** Roamly — <https://github.com/aqn96/roamly>

1. Introduction

A lot of apps really just need to answer one question: where did the user actually go? Fitness trackers, food-delivery apps, navigation, travel apps and similar apps all record a GPS trail. I ran into this building Roamly, a travel app where you “contribute” the routes you walk to unlock recommendations from other travelers. Recording an accurate path is basically the whole point of the app, so I had to figure out how to do it properly.

And it’s trickier than it sounds. Grabbing the phone’s location once is easy; *continuously* logging it while the user is off doing something else is the hard part and challenge due to Apple/Samsung having strict security regulations for background applications. You really need two things working together:

1. FusedLocationProviderClient — the thing that gives you a steady stream of GPS fixes.
2. A foreground service — so the phone keeps logging even after the user leaves your app (in Roamly they’re usually navigating in Google Maps while we quietly record in the background).

So what’s the actual tool here?

FusedLocationProviderClient lives in Google Play Services Location (`com.google.android.gms:play-services-location`). The “fused” bit just means it blends GPS, Wi-Fi, cell towers, and the motion sensors into one clean location stream, and it lets you decide how aggressive to be. For example how often it updates, how accurate, how far you have to move before it bothers you through a `LocationRequest`. It’s what Google points you to these days instead of the old `LocationManager`.

A foreground service is just an Android Service that the system promises not to kill, as long as it keeps a notification on screen so the user knows something is running. That’s the functional way to keep doing work for location, music, navigation when your app isn’t in front.

Here’s the catch, and the reason you need both: on its own, FusedLocationProviderClient quietly stops sending updates the second your app goes to

the background (and newer Android versions keep tightening this). Drop it inside a Foreground Service and it just keeps going. That combo: a fused location stream running out of a foreground service, piped up into Jetpack Compose is what this tutorial is about and what makes my app unique as a passive traveling tracker app.

App flow: Login to your account, search a location/place you want to travel, tap “Start Trip” and the app starts logging your GPS path in the background and draws it live; tap “Stop Trip” and it saves the route. This is intentionally simple so that users can intuitively understand how it works.

The rest of the report covers why this isn’t something the course already taught (§2), a little background on the two APIs (§3), what you need before you start (§4), the actual step-by-step build with code and screenshots (§5–6), where to grab the finished app (§7), and a short conclusion (§8).

2. Why this utility is not covered in the course

The eight course modules (docs/week1.pdf–week8.pdf) cover Kotlin, the Activity lifecycle, Jetpack Compose, Navigation, ViewModel/StateFlow, Notifications, runtime permissions, BroadcastReceiver, LazyColumn, and Retrofit networking. What none of them touch is location: neither the Google Play Services Location APIs (FusedLocationProviderClient) nor Android Services / foreground services show up anywhere in the course.

Course Week	Topic	Related but <i>different</i>
Week 6	ViewModel, StateFlow, Notifications, runtime permissions, BroadcastReceiver	Teaches NotificationChannel and ActivityResultContracts.RequestPermission — but only for status-bar alerts, never tied to a Service or to location
Week 8	Retrofit networking	Background work via coroutines, but no Service and no location

So yes, this tutorial leans on two things the course did teach (asking for a runtime permission and making a notification channel), but the actual meat of it — pulling GPS updates from FusedLocationProviderClient inside a Service that runs in the foreground — is all new ground.

3. Background: the two APIs

FusedLocationProviderClient fuses GPS, Wi-Fi, and cell signals into a single location stream. You describe what you want with a LocationRequest (update interval, priority/accuracy, minimum distance) and receive results through a LocationCallback. It lives in the `com.google.android.gms:play-services-location` artifact.

A foreground service is an Android Service promoted to “foreground” with `startForeground(...)`, which requires an ongoing notification. The OS treats it as user-visible work and won’t casually kill it — which is exactly what you want for logging location while the user is off in another app.

The challenge that ties them together: a Service and the Compose UI are separate lifecycles, so we need a way to share the growing list of GPS points between them. We solve that with a process-wide StateFlow (§5.4).

4. Prerequisites

- Android Studio (recent stable), a device/emulator on API 28+.
- A Jetpack Compose project (the course’s standard setup).
- On an emulator, simulate movement with “Extended Controls → Location” or `adb emu geo fix <lon> <lat>`.

5. Step-by-step implementation

5.1 Add the dependency

In the version catalog (`gradle/libs.versions.toml`) and the app `build.gradle.kts`:

```
// libs.versions.toml
playServicesLocation = "21.3.0"
play-services-location = { group = "com.google.android.gms", name = "play-services-location", version.ref = "playServicesLocation" }
```

```
// app/build.gradle.kts – dependencies { }
implementation(libs.play.services.location)
```

5.2 Declare permissions and the service in `AndroidManifest.xml`

```
<!-- GPS route logging -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<!-- Foreground Service keeps logging while another app is in front -->
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission
  android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />
```

```

<!-- Android 13+ runtime permission for the ongoing notification -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<application ...>
    <service
        android:name=".location.TripLocationService"
        android:exported="false"
        android:foregroundServiceType="location" />
</application>

```

FOREGROUND_SERVICE_LOCATION and android:foregroundServiceType="location" are required on Android 14+ for a location-typed foreground service.

5.3 Model a single GPS sample

```

// data/TrackPoint.kt – one recorded GPS sample; a trip is an ordered list of
// these.
data class TrackPoint(
    val latitude: Double,
    val longitude: Double,
    val timestampMs: Long,
)

```

5.4 A process-wide source of truth (StateFlow)

The Service writes points here; the UI observes them. A single object gives the Service and the UI one shared stream without dependency injection:

```

// Location/TripSession.kt
object TripSession {
    private val _points = MutableStateFlow<List<TrackPoint>>(emptyList())
    val points: StateFlow<List<TrackPoint>> = _points.asStateFlow()

    private val _isRecording = MutableStateFlow(false)
    val isRecording: StateFlow<Boolean> = _isRecording.asStateFlow()

    var startElapsedMs: Long = 0L; private set

    fun begin() { _points.value = emptyList(); startElapsedMs =
        SystemClock.elapsedRealtime(); _isRecording.value = true }
    fun addPoint(p: TrackPoint) { _points.update { it + p } } // new
    list -> StateFlow emits
    fun end() { _isRecording.value = false }

    /** Great-circle route length in metres. */
    fun distanceMeters(pts: List<TrackPoint> = _points.value): Double {
        if (pts.size < 2) return 0.0
        var total = 0.0; val out = FloatArray(1)
        for (i in 1 until pts.size) {
            Location.distanceBetween(pts[i-1].latitude, pts[i-1].longitude,

```

```

pts[i].latitude, pts[i].longitude, out)
    total += out[0]
}
return total
}
}

```

5.5 The Foreground Service

This is the heart of the tutorial — a Service that requests location updates and feeds each fix into TripSession, while showing a persistent notification:

```

// location/TripLocationService.kt
class TripLocationService : Service() {

    private lateinit var fusedClient: FusedLocationProviderClient

    private val locationCallback = object : LocationCallback() {
        override fun onLocationResult(result: LocationResult) {
            result.locations.forEach { loc ->
                TripSession.addPoint(TrackPoint(loc.latitude, loc.longitude,
                    System.currentTimeMillis()))
            }
        }
    }

    override fun onCreate() {
        super.onCreate()
        fusedClient = LocationServices.getFusedLocationProviderClient(this)
    }

    @SuppressWarnings("MissingPermission") // permission verified by the UI
    before starting the service
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int):
    Int {
        startAsForeground()
        val request =
        LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 4000L)
            .setMinUpdateIntervalMillis(2000L)
            .setMinUpdateDistanceMeters(5f)
            .build()
        fusedClient.requestLocationUpdates(request, locationCallback,
        Looper.getMainLooper())
        return START_STICKY // restart if the OS kills
        us
    }

    override fun onDestroy() {
        fusedClient.removeLocationUpdates(locationCallback)
    }
}

```

```

        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null

    private fun startAsForeground() {
        val channel = NotificationChannel("trip_recording", "Trip Recording",
NotificationManager.IMPORTANCE_LOW)

getServiceService(NotificationManager::class.java).createNotificationChannel(c
hannel)
        val notification = NotificationCompat.Builder(this, "trip_recording")
            .setContentTitle("Roamly is recording your route")
            .setContentText("Logging your path in the background ✨")
            .setSmallIcon(android.R.drawable.ic_menu_mylocation)
            .setOngoing(true)
            .build()
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
            ServiceCompat.startForeground(this, 2001, notification,
ServiceInfo.FOREGROUND_SERVICE_TYPE_LOCATION)
        } else {
            ServiceCompat.startForeground(this, 2001, notification, 0)
        }
    }
}

```

5.6 Request location permission at runtime (Compose)

Before starting the service, ask for permission with the course's rememberLauncherForActivityResult pattern (one of the reused building blocks):

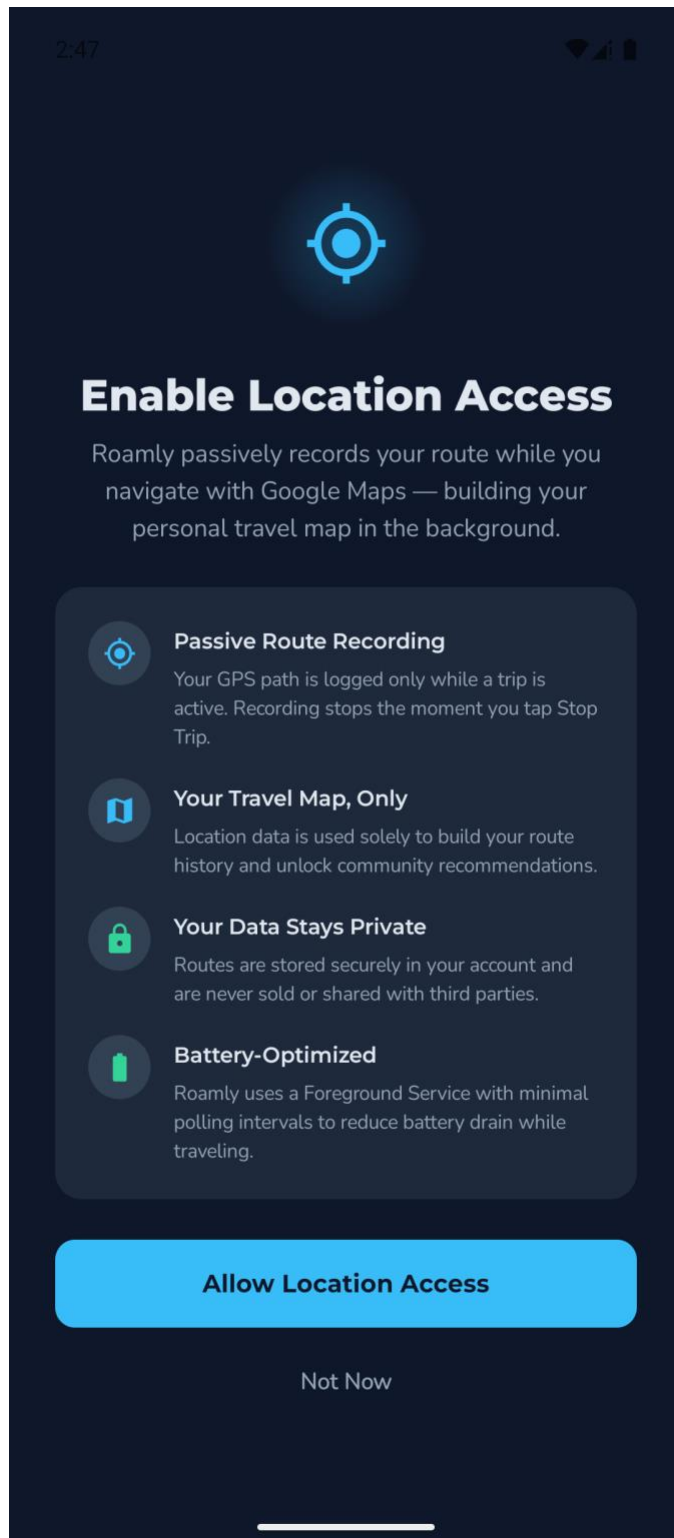
```

// in LocationPermissionScreen.kt
val permissionLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { onAllowClicked() } // proceed once the user
responds

val requested = buildList {
    add(Manifest.permission.ACCESS_FINE_LOCATION)
    add(Manifest.permission.ACCESS_COARSE_LOCATION)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
add(Manifest.permission.POST_NOTIFICATIONS)
}.toArray()

Button(onClick = { permissionLauncher.launch(requested) }) { Text("Allow
Location Access") }

```



Location permission rationale screen

5.7 Start/stop the service from a ViewModel

The ViewModel launches the Foreground Service and exposes the live route + a derived distance as StateFlow (course Topic 6 pattern), so the UI just observes:

```
// ui/screens/trip/TripViewModel.kt
class TripViewModel : ViewModel() {
    val routePoints: StateFlow<List<TrackPoint>> = TripSession.points
    val isRecording: StateFlow<Boolean> = TripSession.isRecording

    val distanceKm: StateFlow<Double> = TripSession.points
        .map { TripSession.distanceMeters(it) / 1000.0 }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), 0.0)

    fun startTrip(context: Context) {
        if (isRecording.value) return
        TripSession.begin()
        ContextCompat.startForegroundService(context, Intent(context,
TripLocationService::class.java))
    }

    fun stopTrip(context: Context) {
        TripSession.end()
        context.stopService(Intent(context, TripLocationService::class.java))
    }
}
```

5.8 Render the live route with a Compose Canvas

The screen observes the ViewModel with collectAsStateWithLifecycle(), starts the service on first composition, and draws the points as a polyline. (Drawing on a Canvas keeps the app on Firebase's free tier — in production this would be a Google Maps Polyline; the recorded data is identical.)

```
@Composable
fun ActiveTripScreen(tripViewModel: TripViewModel = viewModel()) {
    val context = LocalContext.current
    val routePoints by
tripViewModel.routePoints.collectAsStateWithLifecycle()
    LaunchedEffect(Unit) { tripViewModel.startTrip(context) } // begin
    Logging when shown
    // ... draw routePoints as connected lines on a Canvas (normalised to the
box) ...
}

Canvas(modifier = Modifier.fillMaxSize().padding(28.dp)) {
    val minLat = points.minOf { it.latitude }; val maxLat = points.maxOf {
it.latitude }
    val minLng = points.minOf { it.longitude }; val maxLng = points.maxOf {
it.longitude }
```



```

    val offsets = points.map {
        Offset(
            (((it.longitude - minLng) / (maxLng - minLng)) *
size.width).toFloat(),
            ((1.0 - (it.latitude - minLat) / (maxLat - minLat)) *
size.height).toFloat(), // flip Y (north = up)
        )
    }
    for (i in 1 until offsets.size) drawLine(RoamlyElectric, offsets[i-1],
offsets[i], strokeWidth = 10f, cap = StrokeCap.Round)
    offsets.lastOrNull()?.let { drawCircle(RoamlyAurora, radius = 14f, center
= it) }
}

```

6. Running it

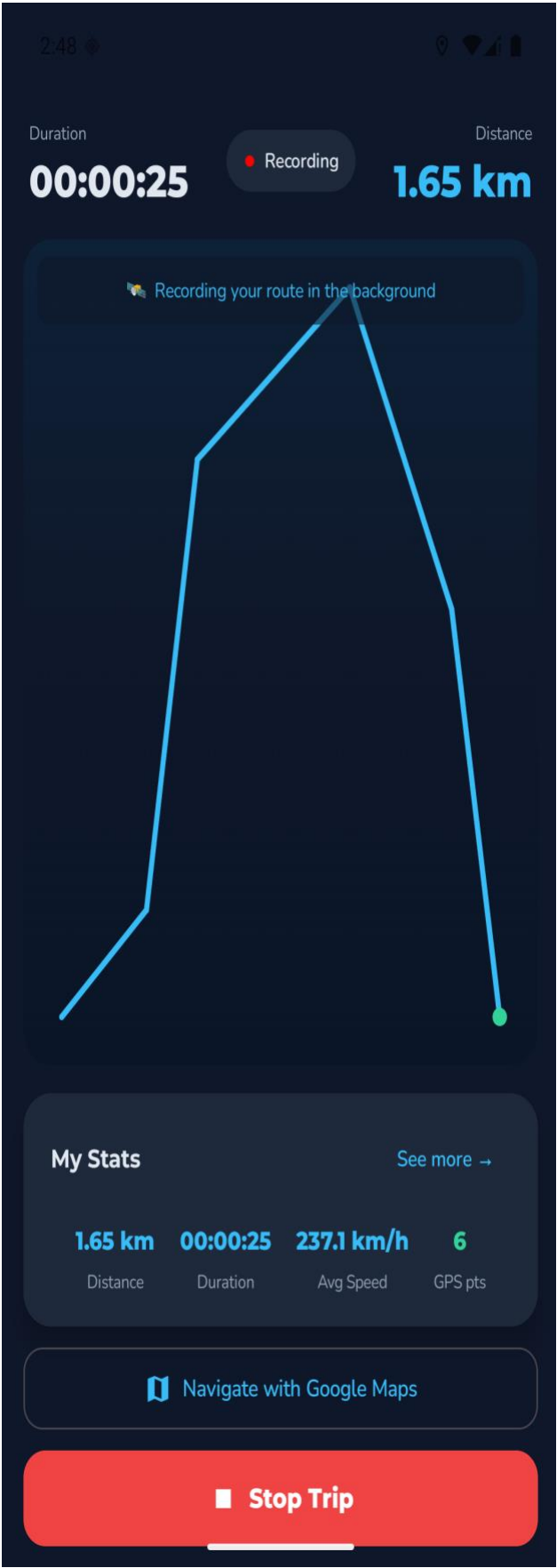
1. Tap “Start Trip” → grant Location (and, on Android 13+, Notification) permission.
2. Move the device (or play a route on the emulator). The blue polyline draws live; duration, distance, and speed update in real time; the ongoing notification confirms background recording.
3. Tap “Stop Trip” → the service stops and the recorded route + stats are shown.

Below you have both screen with the Title:

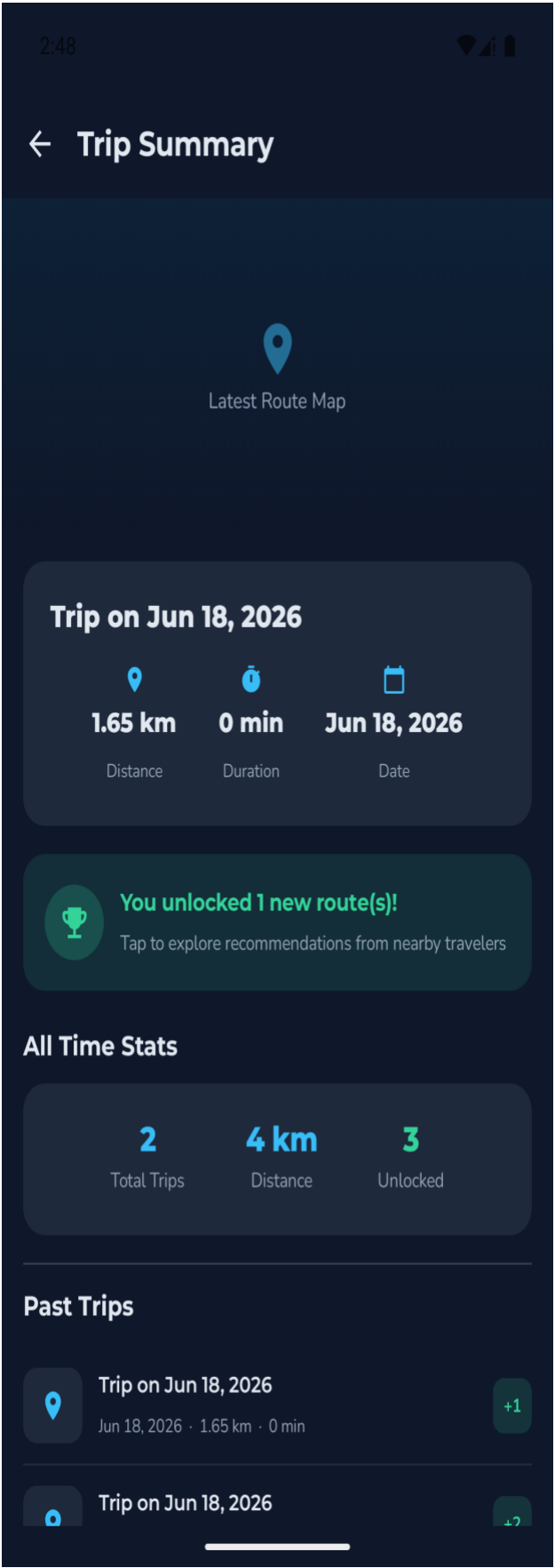
Active recording (live polyline)

Trip summary (saved route)

Active recording (live polyline)



Trip summary (saved route)



NOTE: the route in these screenshots is drawn on a Compose Canvas, not an actual Google Map. That's on purpose. The Google Maps SDK needs a Google Cloud API key and a billing account — Maps gets billed even inside its free monthly credit — and I wanted to keep the whole project on free tiers. It doesn't change what the tutorial is really about: the foreground service is still pulling real GPS fixes from `FusedLocationProviderClient` and recording the same lat/lng points, they're just getting drawn by hand. In a real release you'd keep all of this code and swap the Canvas for a Google Maps Polyline, so you'd get real streets and labels underneath the route.

7. Reference code — a fully working app

The complete, runnable implementation is in Roamly: <https://github.com/aqn96/roamly>

The files that make up this utility:

File	Role
<code>app/src/main/java/com/roamly/app/data/TrackPoint.kt</code>	GPS sample model
<code>app/src/main/java/com/roamly/app/location/TripSession.kt</code>	Shared <code>StateFlow</code> source of truth
<code>app/src/main/java/com/roamly/app/location/TripLocationService.kt</code>	The Foreground Service + <code>FusedLocationProviderClient</code>
<code>app/src/main/java/com/roamly/app/ui/screens/trip/TripViewModel.kt</code>	Start/stop + derived distance
<code>app/src/main/java/com/roamly/app/ui/screens/trip/ActiveTripScreen.kt</code>	Live route Canvas + stats
<code>app/src/main/java/com/roamly/app/ui/screens/home/LocationPermissionScreen.kt</code>	Runtime permission request
<code>app/src/main/AndroidManifest.xml</code>	Permissions + service declaration

To run: clone the repo, add your own `google-services.json` to `app/`, open in Android Studio, and run on an API 28+ device/emulator (see the repo `README.md`).

8. Conclusion

So that's the whole thing: recording a GPS route in the background by pairing two pieces the course never touched — `FusedLocationProviderClient` for a battery-friendly location stream, and a foreground service to keep it running when the app isn't on screen. If there's one idea worth taking away, it's the decoupling: the service and the UI never talk to each other directly. They both go through one process-wide `StateFlow` (`TripSession`), and that's

what lets the route flow cleanly into a ViewModel and onto the screen. It's the sensing backbone of Roamly's whole "contribute a trip to unlock recommendations" idea, and the same setup works for pretty much any app — fitness, delivery, navigation — that needs to answer "where did the user actually go?"